Компактные таблицы пересылки для быстрого поиска маршрутов

Микаэль Дегермарк, Андрей Бродник[3], Сванте Карлссон и Стивен Пинк[2,4]
micke@cdt.luth.se, Andrej.Brodnik@IMFM.Uni-Lj.SI, svante@sm.luth.se, steve@sics.se
Кафедра компьютерных наук и электротехники, Технологический университет Лулео, S-971 87 Лулео, Швеция
[2]Центр дистанционных технологий (CDT), Лулео, Швеция • [3]Институт математики, физики и механики, Любляна, Словения • [4]Шведский институт компьютерных наук (SICS), Киста, Швеция

АННОТАЦИЯ

В течение долгого времени в сетевом сообществе предполагалось, что программный поиск IP-маршрутов со скоростью, достаточной для гигабитных каналов, невозможен. Поиск IP-маршрута должен находить запись с наибольшим совпадающим префиксом (*longest matching prefix*) — задача, которая, как считалось ранее, требует аппаратной поддержки при частоте запросов в миллионы операций в секунду. Мы представляем структуру данных таблицы пересылки, спроектированную для быстрого поиска маршрутов. Таблицы пересылки достаточно малы, чтобы целиком помещаться в кэш обычного процессора общего назначения. Находясь в кэше, таблица позволяет процессору Pentium Pro 200 МГц или Alpha 21164 333 МГц выполнять несколько миллионов поисков в секунду. Это означает, что проведение полного поиска маршрута для каждого IP-пакета на гигабитных скоростях технически осуществимо без специализированного аппаратного обеспечения. Таблицы пересылки исключительно компактны: крупная таблица с 40 000 маршрутных записей может быть сжата до 150–160 Кбайт. Поиск обычно требует менее 100 инструкций на Alpha, совершая восемь обращений к памяти и считывая суммарно лишь 14 байт.

1. Введение

В течение некоторого времени сетевое сообщество исходило из допущения, что полноценный программный поиск IP-маршрутов на микропроцессорах общего назначения со скоростью, достаточной для гигабитной маршрутизации, невозможен. Более того, высказывались мнения, что поиск IP-маршрутов нельзя реализовать быстро и с низкими затратами даже в специализированном аппаратном обеспечении [23].

Мы представляем структуру таблицы пересылки, обеспечивающую быстрый программный поиск IP-маршрутов. Пессимистичные расчеты на базе экспериментальных данных показывают, что процессоры Pentium Pro и Alpha 21164 способны выполнять не менее двух миллионов полных поисков в секунду. Никакая локальность трафика при этом заранее не предполагается.

IP-маршрутизаторы выполняют поиск по таблице маршрутизации для определения того, куда должны пересылаться IP-датаграммы. Результатом операции является следующий узел (*next hop*) на пути к получателю. Запись в таблице концептуально представляет собой префикс произвольной длины с ассоциированной информацией о следующем узле. Поиск обязан найти запись с наибольшим совпадающим префиксом.

Убеждение в том, что поиск IP-маршрутов по своей природе является медленной и сложной операцией, привело к лавинообразному появлению технологий, призванных избежать его выполнения. Разнообразные технологии коммутации канального уровня под IP, методы сквозного обхода уровня IP [15], [19], [20] и разработка альтернативных сетевых уровней на основе технологий виртуальных каналов (таких как ATM) в определенной степени стали следствием стремления избежать поиска IP-маршрутов.

Использование коммутирующих канальных уровней и архитектур коммутации потоков или меток под уровнем IP привносит в сеть избыточную сложность. Коммутация канального уровня и маршрутизация уровня IP выполняют одни и те же функции, поэтому сеть была бы значительно проще, если бы в ней использовалась только одна из этих технологий.

Большинство современных маршрутизаторов используют кэширование, при котором маршрутные записи для недавно использованных адресов назначения сохраняются в кэше. Данный метод опирается на наличие достаточной локальности в трафике, чтобы доля попаданий в кэш была достаточно высока, а затраты на честный поиск амортизировались на несколько пакетов. В прошлом эти методы работали успешно. Однако стремительный рост Интернета увеличивает требуемый размер адресных кэшей, из-за чего аппаратные кэши рискуют стать экономически неоправданными.

Традиционные реализации таблиц маршрутизации используют разновидность деревьев Патриция [13] — структуры данных, изобретенной почти тридцать лет назад, дополненной модификациями для сопоставления по наибольшему префиксу. Применяя современные результаты теории алгоритмов, производительность поиска маршрутов можно повысить на порядки по сравнению с деревьями Патриция.

Прямолинейная реализация деревьев Патриция для задач маршрутизации (например, в стеке NetBSD 1.2) расходует по 24 байта на каждый лист и внутренний узел. При 40 000 записей одна лишь древовидная структура занимает почти 2 мегабайта, а в идеально сбалансированном дереве для поиска записи требуется обойти 15 или 16 узлов. В некоторых случаях из-за правила наибольшего префикса приходится просматривать дополнительные узлы, так как первоначальный спуск не гарантирует нахождения нужного листа. Существуют оптимизации, снижающие размер дерева, однако структура остается слишком громоздкой и требует чрезмерного числа дорогих обращений к памяти. Короче говоря, таблицы маршрутизации были слишком велики для накристальных кэшей, а внешние обращения к DRAM чересчур медлительны для гигабитных скоростей.





Рис. 1: Маршрутизатор с движками пересылки (слева). Рис. 2: Маршрутизатор с вычислениями на интерфейсах (справа).

В оставшейся части статьи представлена структура данных, представляющая таблицы маршрутизации в сверхкомпактной форме и требующая минимум обращений к памяти. Для крупнейших таблиц опорных узлов Сети [21], [22] ее размер составляет 150–160 Кбайт. Она целиком помещается во вторичный кэш Pentium Pro и почти целиком — во вторичный кэш Alpha 21164. Поиск на Alpha требует обычно менее 100 инструкций, 8 обращений к памяти и чтения 14 байт. При префиксах длиннее 28 бит (крайне редких) требуется еще 50 инструкций, 4 обращения и 7 байт. Находясь во вторичном кэше, процессоры Alpha и Pentium Pro способны выполнять свыше двух миллионов поисков в секунду, что при среднем размере пакета 1000 бит (125 байт) эквивалентно пропускной способности более 2 Гбит/с.

2. Таблицы маршрутизации и таблицы пересылки

Архитектура маршрутизатора схематично показана на рисунке 1. Входные интерфейсы передают заголовки пакетов движкам пересылки через фабрику коммутации. Движки вычисляют исходящий интерфейс и возвращают ответ входному интерфейсу, который отправляет пакет дальше. Единственная задача движка — обработка заголовков пакетов. Все остальные функции (маршрутные протоколы, резервирование ресурсов, пакеты с опциями, администрирование) возложены на сетевой процессор (пример — BBN Multigigabit router [17]). Альтернативная архитектура (рисунок 2) размещает процессоры на самих интерфейсах (маршрутизаторы GRF от Ascend).

Оба варианта используют локальную копию — таблицу пересылки, загружаемую из сетевого процессора. Загружать новую таблицу при каждом единичном апдейте не требуется: сходимость протоколов занимает минуты, поэтому таблица может быть слегка устаревшей (*stale*) и обновляться не чаще раза в секунду [6]. Сетевому процессору нужна динамическая таблица для быстрых изменений, тогда как таблица пересылки оптимизируется исключительно под чтение.

3. Цели проектирования и параметры

Главной целью являлась минимизация времени поиска за счет одновременного уменьшения двух параметров: *числа обращений к памяти* и *размера структуры данных*. Обращения к памяти — основное узкое место. Компактная структура целиком удерживается в процессорном кэше, обращаясь к данным на порядки быстрее, чем к медленной памяти DRAM. Локальность трафика удержит горячие узлы в кэше, а для внешнего хранения становится возможным использовать дорогую, но сверхбыструю память SRAM. Вторичные цели: минимум инструкций и естественное выравнивание данных во избежание тяжелых битовых извлечений.

Анализ реальных таблиц показал, что в них содержится менее 60 уникальных next-hop даже при 40 000 маршрутов. Все записи с идентичным next-hop разделяют маршрутные атрибуты. Структура рассчитана на поддержку до 2^{14} (16K) различных next-hop. Если их менее 256, под индекс достаточно одного байта, что сделает таблицу еще компактнее.

4. Структура данных

Таблица пересылки представляет собой дерево из трех уровней. Поиск на одном уровне требует от 1 до 4 обращений к памяти. Максимум обращений — 12, однако на практике абсолютное большинство поисков разрешается за 1–2 уровня (8 или менее обращений).

32
листьев (IP-адреса)
Рис. 3: Полное пространство
e1 e2 диапазон r Рис. 4: Диапазоны адресов
→ Рис. 5: Расширение дерева
Уровень 1 (глубина 16) Уровень 2 (глубины 17–24) Уровень 3 (глубины 25–32) Рис. 6: Три уровня структуры
Модель адресного пространства, диапазоны префиксов, расширение до полного дерева и трехуровневая схема.

Представим бинарное дерево, покрывающее все пространство IP-адресов (рис. 3). Его высота равна 32, а число листьев — 2^{32}. Префикс определяет путь к узлу. Все листья в его поддереве маршрутизируются согласно этой записи. При перекрытии действует правило longest match: на отрезке *r* запись *e1* экранируется записью *e2* (рис. 4). Префиксное дерево дополняется до полного: каждый узел имеет либо два потомка, либо ни одного (рис. 5). Добавленные листья наследуют маршрут ближайшего предка. Это позволяет разбить адреса на диапазоны с длинами, кратными степеням двойки.

Уровень 1 покрывает дерево до глубины 16, уровень 2 — глубины 17–24, уровень 3 — 25–32 (рис. 6). Ветви, уходящие глубже 16, описываются чанками (*chunks*) уровня 2, а глубже 24 — чанками уровня 3.

4.1. Основной результат

Полное бинарное дерево высоты h можно представить, используя **1 бит на возможный лист** на глубине h, плюс **один базовый индекс на 64 листа**, плюс данные в листьях. При $h > 6$ размер в байтах дерева с l листьями и размером информации d равен:

$$Размер = 2^{h-3} + b \times 2^{h-6} + l \times d \qquad (1)$$

где b — размер базового индекса в байтах. При двухбайтовых индексах чанк высоты 8 требует 40 байт плюс данные листьев, а дерево высоты 16 — 10 Кбайт плюс данные листьев. Для адресации требуется константная таблица на 5408 байт (или 1352 байта ценой дополнительных процессорных инструкций).

4.2. Уровень 1 структуры данных

Первый уровень представляет собой узел с 64K потомками и покрывает дерево до глубины 16. Срез на глубине 16 описывается битовым вектором: 2^{16} бит = 64 Кбит = 8 Кбайт. Старшие 16 бит IP-адреса служат индексом в этом векторе.

Головы (Heads). Бит вектора устанавливается в единицу, если на глубине 16 есть узел или если дерево имеет лист на меньшей глубине (выставляется младший бит интервала). Все остальные биты равны нулю. Бит может быть: (1) *корневой головой (root head)* — префикс уходит глубже 16 (биты 6, 12, 13 на рис. 7); (2) *истинной головой (genuine head)* — лист на глубине ≤ 16 (биты 0, 4, 7, 8, 14, 15); (3) *нулем (member)* — адрес внутри диапазона листа (биты 1, 2, 3, 5, 9, 10, 11). Элементы-нули наследуют маршрут ближайшей меньшей головы. Вектор разбит на 4096 16-битных масок.



Рис. 7: Срез дерева и битовый вектор (R — root heads)

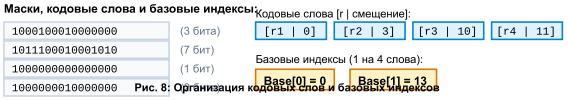


Рис. 8: Организация кодовых слов и базовых индексов

Битовый срез глубины 16, кодирование групп указателей через смещения и базовые адреса.

Информация голов. Для истинных голов хранится индекс в таблице next-hop, а для корневых голов — индекс чанка 2-го уровня. Информация упакована в массив 16-битных указателей: 2 бита задают тип указателя, а 14 бит — индекс таблицы next-hop либо массива чанков уровня 2. Количество указателей в маске равно числу установленных в ней бит.

Поиск группы указателей (рис. 8). Структура состоит из массива кодовых слов (по одному на маску) и массива базовых индексов (один на 4 слова). Кодовое слово состоит из 10-битного значения (*r1, r2, ...*) и 6-битного смещения. Смещение указывает, сколько указателей нужно пропустить. Через каждые 4 слова смещение может превысить 6 бит, поэтому используется 16-битный базовый индекс. Первые 12 бит IP-адреса индексируют массив кодовых слов, а первые 10 бит — массив базовых индексов.

Таблица отображения (Maptable). Значение *r* является индексом в предвычисленной таблице *maptable*, сопоставляющей номер бита в адресе со смещением внутри группы указателей. Так как маски порождены полным деревом префиксов, из 2^{16} комбинаций возможны лишь немногие. Число ненулевых масок *a(n)* длины 2^n определяется рекуррентным соотношением:

$$a(0) = 1, \qquad a(n) = 1 + a(n-1)^2 \qquad (2)$$

Для длины 16 число вариантов составляет $a(4) + 1 = 678$ (с учетом нулевой маски). Для адресации достаточно 10 бит. Таблица *maptable* константна, генерируется один раз и разделяется всеми тремя уровнями.

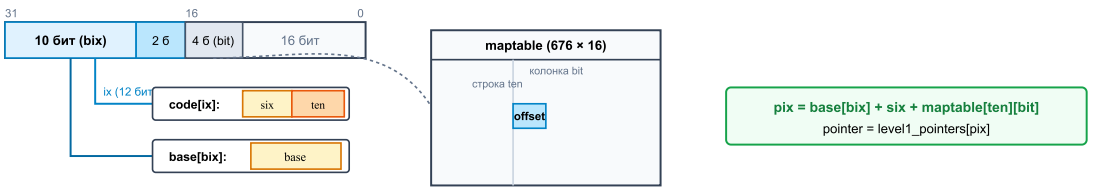


Рис. 9: Аппаратная схема вычисления индекса указателя в массиве первого уровня.

Рис. 10. Алгоритм поиска на первом уровне: `ix := старшие 12 бит IP-адреса; bix := старшие 10 бит IP-адреса; bit := младшие 4 из 16 старших бит; codeword:= code[ix]; ten := 10 бит из codeword; six := 6 бит из codeword; pix := base[bix] + six + maptable[ten][bit]; pointer := level1_pointers[pix];`

Поиск выполняется элементарно: несколько битовых сдвигов, чтение массивов и сложение (рис. 9, 10). Инструкций умножения и деления не требуется. Всего считывается 7 байт: 2 байта кодового слова, 2 байта базы, 1 байт (фактически 4 бита) из *maptable* и 2 байта указателя. Объем первого уровня: 8 Кб слов + 2 Кб баз + массив указателей.

4.2.1. Оптимизации на уровне 1

Если маска нулевая или содержит один бит, указатель гарантированно ведет в таблицу next-hop. Такие указатели кодируются прямо в кодовом слове, что исключает строки 0 и 1 из *maptable*, сокращая ее до 676 записей. Если поле $ten > 675$, слово кодирует прямой индекс: 6 бит смещения служат младшими битами индекса, а $(ten - 676)$ — старшими. Это позволяет адресовать до $(1024 - 676) \times 2^6 = 22\,272$ записей next-hop (с запасом под наши цели в 16K). Оптимизация исключает три обращения к памяти для префиксов глубины ≤ 12 ценой одной инструкции сравнения и условного перехода.

4.3. Уровни 2 и 3 структуры данных

Уровни 2 и 3 состоят из чанков. Чанк покрывает поддерево высоты 8 и может содержать до $2^8 = 256$ голов. Корневая голова на уровне $n-1$ указывает на чанк уровня n. Существует три типа чанков:

1. Разреженные (sparse, 1–8 голов): массив 8-битных индексов голов плюс восемь 16-битных указателей (всего 24 байта). Значения отсортированы по убыванию. Сначала проверяется 4-е значение для локализации диапазона, затем линейным сканированием находится нужный элемент (первый $\leq$ ключа поиска). Считывается не более 7 байт.

2. Плотные (dense, 9–64 голов): аналогичны уровню 1, но требуют лишь один базовый индекс на все 16 кодовых слов (6-битное смещение покрывает все 64 указателя). Всего 34 байта структуры плюс от 18 до 128 байт на указатели.

3. Сверхплотные (very dense, 65–256 голов): аналогичны уровню 1: 16 кодовых слов и 4 базовых индекса (40 байт плюс 130–512 байт указателей).

4.3.1. Оптимизации на уровнях 2 и 3

Плотные и очень плотные чанки оптимизируются аналогично уровню 1. В разреженных чанках последовательные головы объединяются (*merge*) и представляются наименьшей, если их next-hop совпадают. Чанк признается разреженным, если после слияния остается ≤ 8 голов. Это смещает распределение в сторону компактных разреженных блоков и сокращает размер таблицы на 5–15%.

4.4. Пределы роста в текущей архитектуре

Структура допускает значительный рост числа маршрутов. Архитектура имеет три ограничения:

1. Количество чанков каждого типа: ограничено величиной $2^{14} = 16\,384$ на уровень. Таблица 1 показывает, что это примерно в 16 раз больше, чем требовалось крупнейшим таблицам 1997 года.

2. Число указателей на уровнях 2 и 3: ограничено размером базовых индексов (запас роста в 3–5 раз). При превышении базовые указатели расширяются до 3 байт, что увеличит плотный чанк лишь на 3%, а очень плотный — на 10%.

3. Количество уникальных next-hop: лимит в 16 384 адреса. При его превышении часть указателей нельзя будет кодировать напрямую в словах, что незначительно увеличит память.

5. Измерения производительности

Для тестов были собраны реальные таблицы BGP с сайтов проектов IPMA [22] и Routing Arbiter [21]. Дампы представляют собой ежедневные снимки крупнейших узлов межсоединения Сети.

5.1. Размер таблицы пересылки

В таблице 1 представлены данные по таблицам пересылки, построенным для различных точек обмена трафиком. Время построения замерялось на Alpha 21164 333 МГц под управлением DEC OSF1.

Точка обмена	Дата	Маршрутов	Листьев	Next-hops	Размер	Сборка	Sparse	Dense	Dense+	Lvl 3
Mae East	9 янв 1997	32 732	58 714	56	**160 Кб**	99 мс	1199	587	186	2
Mae East	21 окт 1996	38 141	36 607	50	**148 Кб**	91 мс	1060	593	149	4
Sprint	1 янв 1997	21 797	43 513	17	**123 Кб**	72 мс	988	483	98	3
PacBell	28 янв 1997	18 308	33 250	2	**99 Кб**	49 мс	873	357	67	0
Mae West	1 янв 1997	12 049	28 273	51	**86 Кб**	46 мс	775	312	42	3
AADS	4 янв 1997	1 109	5 670	12	**28 Кб**	11 мс	320	38	0	2

Таблица 1: Данные генерации таблиц пересылки для различных узлов Интернета.

Таблица 1 наглядно показывает, что таблицы генерируются чрезвычайно быстро: при частоте обновления в 1 Гц расходуется менее одной десятой вычислительной мощности процессора Alpha. Крупные таблицы не помещаются целиком в 96 Кб L2-кэша Alpha, однако наличие небольшого объема быстрой SRAM в L3-кэше способно свести к минимуму потери при промахах. Удельный расход памяти составляет не более 5.6 байта на префикс, причем более половины этого объема занимают указатели (в таблице Sprint 33 469 указателей занимают более 65 Кб).

Процессор	Такт	Первичный кэш (L1)	Вторичный кэш (L2)	Третичный кэш (L3)
Alpha 21164	3 нс (333 МГц)	8 Кбайт, задержка 6 нс (2 такта)	96 Кбайт, задержка 24 нс (8 тактов)	2 Мбайт, задержка 72 нс
Pentium Pro	5 нс (200 МГц)	8 Кбайт, задержка 10 нс (2 такта)	256 Кбайт, задержка 30 нс (6 тактов)	—

Таблица 2: Параметры задержек и емкости кэш-памяти тестовых микропроцессоров.

5.2. Производительность поиска

Функция поиска написана на C и скомпилирована gcc. gcc генерирует код, требующий около 50 инструкций Alpha и 35–45 инструкций Pentium Pro на один уровень. Время измерялось чтением счетчика тактов. Поиск выполнялся дважды подряд: первый проход прогревал кэш, а второй замерял чистое время выполнения в кэше. Так как в уровне 3 находится мизерное число чанков (Таблица 1), подавляющее большинство пакетов находят маршрут за 1–2 уровня (8 обращений к памяти).



Рис. 11: Распределение времени поиска (Alpha 21164)

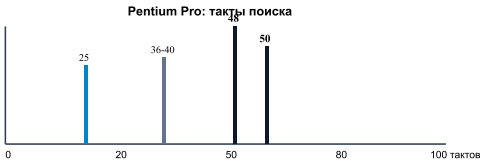


Рис. 12: Распределение времени поиска (Pentium Pro)

Гистограммы распределения задержек поиска при обращении к структуре данных.

Alpha 21164. Быстрейшие поиски требуют 17 тактов (прямое кодирование next-hop в кодовом слове уровня 1). Всплеск на 41 такте соответствует нахождению next-hop в указателе уровня 1. Пики на 52–72 тактах — просмотр разреженных чанков уровня 2, а огромные пики на 75 и 83 тактах — плотные и сверхплотные чанки (рис. 11). Разница между чтением L1 и L2 составляет $8 - 2 = 6$ тактов. Худший случай для 2 уровней требует не более $100 + 8 \times 6 = 148$ тактов (444 нс), что гарантирует **не менее 2.2 миллиона поисков в секунду**.

Pentium Pro. Быстрейший поиск занимает 11 тактов. Всплеск 25 тактов — разрешение на уровне 1. Разреженные чанки группируются на 36–40 тактах, а плотные и сверхплотные требуют 48 и 50 тактов (рис. 12). Разница доступа к L1 и L2 — 4 такта (20 нс). Худший случай: $69 + 8 \times 4 = 101$ такт (505 нс), что обеспечивает **не менее 2.0 миллионов поисков в секунду**.

6. Масштабируемость

Число инструкций не зависит от общего размера таблицы маршрутизации, пока она помещается в кэше. Базовый размер структуры — от 15.3 Кб, далее рост линеен числу листьев дерева. Использование O()-нотации здесь некорректно, так как 32-битное пространство IP конечно. Алгоритм построения таблицы линеен числу префиксов и выполняется за один проход. Для 128-битных адресов IPv6 алгоритм адаптируется динамической настройкой глубины чанков.

7. Связанные работы и альтернативы

Динамические префиксные боры [5] хранят по 5 указателей на узел и выполняют поиск за 6–13 мкс (на порядок медленнее нашего метода). Кэширование адресов [7] при современном трафике требует громоздких многотысячных кэшей. Аппаратные решения на базе ATM, коммутация тегов и потоков [14], [15] и архитектуры IP/ATM [19], [20] создавались лишь для обхода софтовой медлительности IP ценой колоссального усложнения протоколов. Специализированные процессоры (IBM [1]) негибки, а память CAM [9] чрезвычайно дорога. В проекте BBN [17], [18] процессоры Alpha используются как движки пересылки, но опираются на LRU-кэш адресов, сильно зависящий от локальности трафика.

8. Обсуждение и заключение

Даже с учетом дополнительных 100 инструкций на разбор остальных полей IP-заголовка процессоры Alpha и Pentium Pro способны самостоятельно обрабатывать миллион пакетов в секунду. Алгоритм прямо переносится в специализированное железо: выборки из базового массива и кодового слова независимы и могут выполняться параллельно, а конвейеризация сократит задержку до времени обхода одного уровня.

Заключение. Мы доказали практическую осуществимость полного поиска IP-маршрутов на гигабитных скоростях без специализированного аппаратного обеспечения. Компактная таблица пересылки эффективно масштабируется, гарантируя миллионы операций поиска в секунду на серийных процессорах общего назначения.

Литература

1. [1] Asthana A., Delph C., Jagadish H.V., Krzyzanowski P. Towards a gigabit IP router // Journal of High Speed Networks, 1(4):281–288, 1993.
2. [2] Brodnik A., Munro J.I. Membership in a constant time and a minimum space // Proc. 2nd ESA, LNCS 855, pp. 72–81, 1994.
3. [3] Brodnik A., Munro J.I. Neighbours on a grid // Proc. 5th SWAT, LNCS 1097, pp. 307–320, 1996.
4. [4] Deering S., Hinden R. Internet Protocol, Version 6 (IPv6) Specification // RFC 1883, 1996.
5. [5] Doeringer W., Karjoth G., Nassehi M. Routing on longest-matching prefixes // IEEE/ACM Trans. Networking, 4(1):86–97, 1996.
6. [6] Stanford University Workshop on Fast Routing and Switching, Dec 1996.
7. [7] Feldmeier D.C. Improving gateway performance with a routing-table cache // Proc. IEEE INFOCOM, 1988.
8. [8] Hinden R. IP Next Generation Home Page, 1996.
9. [9] McAuley A.J., Francis P. Fast routing table lookup using CAMs // Proc. IEEE INFOCOM, pp. 1382–1391, 1993.
10. [10] McVoy L. lmbench home page, 1996.
11. [11] McVoy L., Staelin C. lmbench: Portable tools for performance analysis // USENIX Winter Conf., 1996.
12. [12] Mehlhorn K., Näher S., Alt H. A lower bound on the complexity of union-split-find // SIAM J. Comput., 17(1):1093–1102, 1988.
13. [13] Morrison D.R. PATRICIA Practical Algorithm to Retrieve Information Coded In Alfanumeric // J. ACM, 15(4):514–534, 1968.
14. [14] Newman P., Edwards W.L., et al. Ipsilon Flow Management Protocol Specification for IPv4 // RFC 1953, 1996.
15. [15] Newman P., Lyon T., Minshall G. Flow labeled IP: a connectionless approach to ATM // Proc. IEEE INFOCOM, 1996.
16. [16] Nilsson S. Radix Sorting & Searching // PhD thesis, Dept. of CS, Lund University, 1996.
17. [17] Partridge C., Carvey P., Burgess E., et al. A fifty gigabit per second IP router // IEEE/ACM Trans. Networking, 1998.
18. [18] Partridge C. Gigabit networking // Addison-Wesley, Reading, MA, 1993.
19. [19] Parulkar G., Schmidt D.C., Turner J. IP/ATM: A strategy for integrating IP with ATM // CCR, 25(4):49–58, 1995.
20. [20] Parulkar G., Schmidt D.C., Turner J.S. GIPR: a gigabit IP router // Proc. Gigabit Networking Workshop, 1995.
21. [21] The Routing Arbiter Project. Internet routing and network statistics, 1996.
22. [22] Michigan Univ., Merit Network. IPMA Project: Internet Performance Measurement and Analysis, 1997.
23. [23] Washington University Workshop on Integration of IP and ATM, Nov 1996.